ICICI Bank Payment Gateway

Initiate Sale & Transaction APIs — Merchant Integration Reference (UAT)

This document covers the Sale transaction flow of the ICICI Bank Payment Gateway: generating the Initiate Sale request, redirecting the customer for payment, receiving the outcome on the return URL, confirming the transaction independently, and handling Refunds, Settlement Status and Generate QR.

Quick Integration Flow — Step by Step

At a high level, integrating the Initiate Sale API involves the following steps. Each step is explained in detail later in this document.

1. Generate the secureHash

Before calling any API in this document, generate the secureHash for the request payload and include it in the request body. The hash-generation method (Section 2.1) is common to every API call described here — Initiate Sale, Status Check, Refund, Settlement Status and Generate QR all use the same method.

2. Build the redirecting URL and send the customer to the bank page

Call Initiate Sale with the secureHash included. On a successful response, construct the redirecting URL using the redirectURI and tranCtx values returned in the response (Section 2.2), and redirect the customer's browser to it to complete payment.

3. Receive the outcome on the return URL, or via webhook

Once the customer completes payment, the bank posts the outcome to the merchant's returnURL as an HTML form POST (Section 2.3). In parallel (or as a fallback if the customer does not return to the merchant site), the same outcome is delivered server-to-server as a Payment Advice webhook (Section 3.1). Whitelist the bank's webhook source and use the advice payload to update the order status on success.

4. Confirm with the Status Check API

Regardless of what the return URL or webhook reports, always confirm the final transaction status independently using the Status Check API (Section 1.2) before treating an order as paid.

Syllabus

1. APIs — Requests & Responses
    1. Initiate Sale
    2. Status Check
    3. Refund API
    4. Settlement Status
    5. Generate QR
2. Description about the API
    1. How to Create the Secure Hash Key
    2. How to Create the Redirecting URL
    3. How to Create the POST HTML Page (Return URL)
3. Webhooks (Payment & Settlement Advice)
    1. Payment Advice
    2. Settlement Advice
4. Error Codes
    1. Quick Status Reference
    2. Full Error / Response Code List
5. Credentials (UAT)
6. Test Card & Test Payment Details (UAT)
7. API URLs (UAT & Production)
8. FAQs

1. APIs — Requests & Responses

All endpoints below are UAT (sandbox) URLs. Replace with production hosts before go-live (see Section 6).

1.1 Initiate Sale

Endpoint: POST https://pgpayuat.icicibank.com/tsp/pg/api/v2/initiateSale

Request

```
{
  "aggregatorID": "A100000000007164",
  "amount": "1.00",
  "currencyCode": "356",
  "customerEmailID": "test@test.akk",
  "customerMobileNo": "9999999999",
  "customerName": "test",
  "merchantId": "100000000007164",
  "merchantTxnNo": "4560012_SANDBOX",
  "payType": 0,
  "transactionType": "SALE",
  "txnDate": "20260720150850",
  "udfFields": {
    "udf23": "AMCCode28-AMCName-Folio28-UserTxnNo28-SchemeCode28-abc28",
    "udf24": "AMCCode29-AMCName-Folio29-UserTxnNo29-SchemeCode29-abc29"
  }
}
```

Response

```
{
  "responseCode": "R1000",
  "merchantId": "100000000007164",
  "aggregatorID": "A100000000007164",
  "merchantTxnNo": "4560012_SANDBOX",
  "redirectURI": "https://pgpayuat.icici.bank.in/tsp/pg/api/v2/authRedirect",
  "showOTPCapturePage": "N",
  "generateOTPURI": null,
  "verifyOTPURI": null,
  "authorizeURI": null,
  "tranCtx": "R019f7ed8-19a6-70d1-9720-61ece38b67b7",
  "udfFields": "{...same udfFields, JSON-encoded as string...}",
  "returnURL": "https://test.com/statuspage",
  "secureHash": "c648ebda7eabd08854d8ff786e35ca1f146fbd2a51e454c7a8c5dbf03e934186"
}
```

Note :- After a successful call to Initiate Sale, the response returns a redirectURI and a transaction context token, tranCtx. To take the customer to the bank's payment page, redirect the customer's browser to: {redirectURI}?tranCtx={tranCtx}

Example, using the sample Initiate Sale response: https://pgpayuat.icicibank.com/tsp/pg/api/v2/authRedirect?tranCtx=Ra3c22840-bb8b-4e44-b622-e35928b86858

Note: udfFields is optional — only include the UDF slots your merchant profile is configured for.

1.2 Status Check

Endpoint: POST https://pgpayuat.icicibank.com/tsp/pg/api/command?reqType=JSON

Request

```
{
  "aggregatorID": "A100000000007164",
  "merchantId": "100000000007164",
  "merchantTxnNo": "TX126443453",
```

```
  "originalTxnNo": "4560012_SANDBOX",
  "secureHash": "e0532a4cad316991c0204ff81fa6b6ba458c86f2b823d55aabc03d37eb67e8e6",
  "transactionType": "STATUS"
}
```

Response

```
{
  "txnRespDescription": "Transaction successful",
  "secureHash": "3dec30d34ca821b468afe4c0a77037213d318c1803df00b32d743002a9cc563e",
  "amount": "1.00",
  "txnAuthID": "92485056629",
  "txnResponseCode": "0000",
  "paymentMode": "NB",
  "respDescription": "Request processed successfully",
  "aggregatorID": "A100000000007164",
  "TransmissionDateTime": "20260720145903",
  "responseCode": "000",
  "transactionType": "SALE",
  "acqName": "ICICI Bank",
  "txnStatus": "SUC",
  "merchantId": "100000000007164",
  "merchantTxnNo": "4560012_SANDBOX",
  "paymentDateTime": "20260720145910",
  "txnID": "7700229146709"
}
```

1.3 Refund API

Endpoint: POST https://pgpayuat.icicibank.com/tsp/pg/api/command?reqType=JSON

Request

```
{
  "aggregatorID": "A100000000007164",
  "amount": "1.00",
  "merchantId": "100000000007164",
  "merchantTxnNo": "R12324",
  "originalTxnNo": "4560012_SANDBOX",
  "secureHash": "a1e5b20d2be16c298f9ee92dd5c7133b5b30e0e2d657286c67b5713f1e2edf20",
  "transactionType": "REFUND"
}
```

Response

```
{
  "secureHash": "660c3b0addd98a8581acb2e584bf9395c483804044c57c21ccccc025c16e195a",
  "acqName": "ICICI Bank",
  "merchantId": "100000000007164",
  "respDescription": "Request processed successfully",
  "merchantTxnNo": "R12324",
  "aggregatorID": "A100000000007164",
  "TransmissionDateTime": "20260720145903",
  "responseCode": "R1000",
  "txnID": "7700229146735"
}
```

1.4 Settlement Status

Endpoint: POST https://pgpayuat.icicibank.com/tsp/pg/api/command?reqType=JSON

Note: Uses the SETTLSTATUS command, queried against a settled transaction.

Request

```
{
  "aggregatorID": "A100000000007164",
  "merchantId": "100000000007164",
```

```
  "merchantTxnNo": "wedsxawzsdhhx",
  "originalTxnNo": "85787989eecsx",
  "secureHash": "189cfe438368112b050dfe8c77887095e62a5e68787f4faeeb526a53479fe20d",
  "transactionType": "SETTLSTATUS"
}
```

Response

```
{
  "txnRespDescription": "Transaction successful",
  "secureHash": "3f97878ef2eb76df9659c5ca372d7fa9d7469565682a99926686653ddbd4a0a6",
  "amount": "1.00",
  "txnAuthID": "25749226031",
  "txnResponseCode": "0000",
  "paymentMode": "Card",
  "respDescription": "Request processed successfully",
  "aggregatorID": "A100000000007164",
  "TransmissionDateTime": "20260714094422",
  "paymentInstId": "XXXX XXXX XXXX 0035",
  "responseCode": "000",
  "transactionType": "SALE",
  "acqName": "ICICI Bank",
  "cardNetwork": "VISA",
  "txnStatus": "SUC",
  "paymentSubInstType": "CC",
  "merchantId": "100000000007164",
  "merchantTxnNo": "85787989eecsx",
  "paymentDateTime": "20260714094431",
  "txnID": "7700229105821"
}
```

1.5 Generate QR

Endpoint: POST https://pgpayuat.icicibank.com/tsp/pg/api/generateQR

Note: Request is sent as application/x-www-form-urlencoded, not JSON.

Request (cURL)

```
curl --location 'https://pgpayuat.icicibank.com/tsp/pg/api/generateQR' \
--header 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'merchantId=100000000007164' \
--data-urlencode 'aggregatorID=A100000000007164' \
--data-urlencode 'amount=1.00' \
--data-urlencode 'currency=356' \
--data-urlencode 'emailID=test@gmail.com' \
--data-urlencode 'merchantRefNo=G12345' \
--data-urlencode 'requestType=UPIQR' \
--data-urlencode 'secureHash=a7947a3568b649f58dea463fff809fe5bcc6d20930cedfca2b965c5092f2a0d1'
```

Response

```
{
  "respHeader": {
    "returnCode": 200,
    "desc": "Success",
    "upiRespDesc": "SUC"
  },
  "respBody": {
    "merchantID": "100000000331670",
    "aggregatorID": "100000000331669",
    "merchantRefNo": "G12345",
    "bharatQR": null,
    "upiQR": "upi://pay?pa=IBMSEO000009@icici&pn=UPI channel code testing 1&mc=6012&tr=PGP000000000PHI26401492458
    "expiry": "22/07/2026 16:00:08",
    "serviceChargeUPI": "0.00",
    "serviceChargeBharatQR": null
  }
}
```

2. Description about the API

2.1 How to Create the Secure Hash Key

Every request/response to the ICICI Payment Gateway carries a secureHash field. This hash is an HMAC-SHA256 signature computed over a concatenated string of the request fields, signed using the merchant's shared secret key. It lets the bank (and the merchant, on the response side) verify that the payload was not tampered with in transit.

There is a single method used to generate the hash for every API in this document: sort the request's top-level JSON keys alphabetically (ksort), then concatenate each key's value in that sorted order — scalars are appended as-is, and any nested object/array (e.g. udfFields) is JSON-stringified first. secureHash itself is always excluded from the string. The resulting string is signed with HMAC-SHA256 using the merchant secret key to produce the hex-encoded secureHash.

Reference implementation (JavaScript / Web Crypto API)

```
async function generateSecureHash(requestObj, secretKey) {
  const sorted = Object.keys(requestObj).sort().reduce((o, k) => {
    o[k] = requestObj[k];
    return o;
  }, {});
  let hashText = '';
  Object.keys(sorted).forEach(key => {
    if (key === 'secureHash') return;
    const value = sorted[key];
    if (value !== null && typeof value === 'object') {
      hashText += JSON.stringify(value); // nested object/array -> JSON string
    } else if (value !== undefined && value !== null) {
      hashText += value; // scalar -> appended as-is
    }
  });
  const encoder = new TextEncoder();
  const key = await crypto.subtle.importKey(
    'raw', encoder.encode(secretKey),
    { name: 'HMAC', hash: 'SHA-256' }, false, ['sign']
  );
  const sigBuf = await crypto.subtle.sign('HMAC', key, encoder.encode(hashText));
  return Array.from(new Uint8Array(sigBuf))
    .map(b => b.toString(16).padStart(2, '0')).join('');
}
```

2.2 How to Create the Redirecting URL

After a successful call to Initiate Sale, the response returns a redirectURI and a transaction context token, tranCtx. To take the customer to the bank's payment page, redirect the customer's browser to: {redirectURI}?tranCtx={tranCtx}

Example, using the sample Initiate Sale response:
https://pgpayuat.icicibank.com/tsp/pg/api/v2/authRedirect?tranCtx=Ra3c22840-bb8b-4e44-b622-e35928b86858

• redirectURI is returned in the Initiate Sale response and points to the bank's hosted authorization page.

• tranCtx is the unique transaction context ID generated for that Initiate Sale call and is passed as a query parameter.

• Once the customer completes payment on the bank page, the bank redirects back to the merchant's returnURL (as sent in the Initiate Sale request) with the transaction outcome.

• The final transaction status should always be confirmed independently using the STATUS API rather than relying solely on the return redirect.

2.3 How to Create the POST HTML Page (Return URL)

The merchant's returnURL is not called with a redirect (GET) — the bank posts the payment outcome back to it as an HTML form POST, with content type application/x-www-form-urlencoded. The merchant's returnURL endpoint should therefore be built to accept a standard form POST, read the posted fields as name/value pairs, and then decide what to show the customer.

Step-by-step, for both integration modes:

• Redirection mode (showOTPCapturePage = 'N'): after redirecting the browser to {redirectURI}?tranCtx={tranCtx} and the customer completes authentication/authorization on the issuer's page, the bank's server itself submits a hidden HTML form via POST to the merchant's returnURL — the browser is simply carrying that POST, the merchant does not construct this form.

• Seamless mode (showOTPCapturePage = 'Y'): after a successful authorize call the response is returned directly as JSON to the merchant's server, so no browser POST/redirect is involved for that flow; the merchant still receives payment advice via the webhook mechanism (Section 3.1).

Only the POST body parameters are used for the secureHash verification on the returnURL — any query-string parameters on the returnURL are ignored for hash calculation. The merchant must independently recompute the hash over the received POST fields (Section 2.1 method) and compare it to the secureHash field in the POST body before trusting the result.

Sample auto-submitting form posted to the merchant's returnURL

```
<form id='myForm' name='myForm' action='${merchantReturnURL}' method='post'
  enctype="application/x-www-form-urlencoded">
  <input type="hidden" name="responseCode" value="000" />
  <input type="hidden" name="respDescription" value="SUCCESS" />
  <input type="hidden" name="merchantId" value="M000001" />
  <input type="hidden" name="merchantTxnNo" value="M0099999221" />
  <input type="hidden" name="txnID" value="T1472640294491" />
  <input type="hidden" name="paymentDateTime" value="20160831041454" />
  <input type="hidden" name="paymentID" value="006503" />
  <input type="hidden" name="secureHash"
    value="39867b8ef38a2853aacb84bbf5f927a72b27bdb301365245564f012d20dbc2f2"/>
</form>
<script>document.getElementById('myForm').submit();</script>
```

Note: The merchant's returnURL page should read these POST fields server-side, verify secureHash, then render the appropriate success/failure page to the customer.

3. Webhooks (Payment & Settlement Advice)

In addition to the redirect/response flow, ICICI Payment Gateway can push transaction and settlement status updates directly to the merchant's own server via webhook-style server-to-server calls. The merchant must expose an HTTPS endpoint (an “advice URL”) to receive these; the gateway calls it independently of the browser, so it works even if the customer never returns to the merchant site.

• Any response with HTTP status code 200 is treated as successful delivery of the webhook.

• If delivery fails (a non-200 status code), the webhook can be configured to retry.

• Webhook payloads can be delivered either as a standard form POST (application/x-www-form-urlencoded) or as JSON (application/json), depending on what was agreed during merchant onboarding.

3.1 Payment Advice

Payment Advice notifies the merchant of a transaction status update, most commonly used when the status changes some time after the original online transaction completed. It carries the same fields as the standard Payment Response (Section 1.2).

Form-encoded payload (application/x-www-form-urlencoded)

```
responseCode=0000&respDescription=Transaction%20successful&merchantId=T_05022&merchantTxnNo=638948982&txnID=T00..
```

JSON payload (application/json)

```
{
  "responseCode": "0000",
  "respDescription": "Transaction successful",
  "merchantId": "T_05022",
  "merchantTxnNo": "638948982",
  "txnID": "T002473066701",
  "paymentDateTime": "20210817060032",
  "paymentID": "700788003737"
}
```

The merchant's advice endpoint should simply reply with HTTP 200 once the update has been recorded. Whitelist this source in your firewall/allow-list so the webhook is not blocked.

Sample decoded advice payload — Sale API (real transaction example)

```
{
  "aggregatorID": "A100000000007164",
  "TransmissionDateTime": "20260617155311",
  "txnType": "SALE",
  "oth_charge": 0.03,
  "authCode": "bscfe03bbfe033",
  "bankCode": "9734",
  "customerMobileNo": "9515486031",
  "customerEmailID": "integrationtestpg@gmail.com",
  "addlParam1": "test20260617155230",
  "addlParam2": "test20260617155230",
  "paymentSubInstType": "CC Avenues Test Bank",
  "paymentMode": "NB",
  "amount": "1.00",
  "responseCode": "0000",
  "respDescription": "Transaction successful",
  "merchantId": "100000000007164",
  "merchantTxnNo": "test20260617155230",
  "txnID": "7700228099888",
  "paymentDateTime": "20260617155314",
  "paymentID": "315014545310",
  "secureHash": "75896e950985cd23c2847a45e3c66f1bfeb30b81e8ad7dfceff22af148e51926"
}
```

3.2 Settlement Advice

Settlement Advice notifies the merchant when a transaction has been settled to their bank account. This is always sent as a server-to-server call with a JSON body in the POST request.

Sample Settlement Advice payload (JSON)


```
{
  "aggregatorID": "A001",
  "merchantId": "123243",
  "merchantTxnNo": "RF12937",
  "invoiceNos": "343223",
  "settlementID": "A00121-20171101-010",
  "settlementDate": "20171101",
  "paymentMode": "card",
  "paymentSubInstType": "DC",
  "cardNetwork": "VISA",
  "txnChannel": "WEB",
  "secureHash": "39867b8ef38a2853aacb84bbf5f927a72b27bdb301365245564f012d20dbc2f2"
}
```


Key fields carried in the settlement advice include:

• settlementID — the settlement/credit reference number linked to the payout.

• settlementDate — date of settlement, format YYYYMMDD.

• settlementAccount / settlementAccountIFSC — the account the funds were settled into (only if the merchant is certified to receive these).

• settledAmount — the amount actually settled for the transaction.

• cardNetwork — VISA / MasterCard / AMEX / RUPAY, when paymentMode is CARD.

Note: As with Payment Advice, any HTTP response with status code 200 is treated as a successful delivery of the settlement advice.

4. Error Codes

4.1 Quick Status Reference

Code(s)	Meaning
0 / 00 / 0000	SUC — Success
039	REJ — Rejected
R1000	REQ — Request accepted
DT001	Duplicate transaction

4.2 Full Error / Response Code List

Code	Description
039	DEFAULT_RESPCODE
P0039	REQUEST_NOT_FOUND
P0030	AWAITING_USER_ACTION
P0031	TXN_EXPIRED
P0032	REFUND_RECOVERY_NOT_COMPLETED
P9001	GTWY_CONN_ERR
P9002	PEER_TOOSLOW_ERR
P9003	NO_ROUTE_ERR
P9004	GTWY_NOTFOUND_ERR
P9005	SYS_SHUTDOWN
P9009	HSM_CONN_ERR
P9010	HSM_INTF_TO_ERR
P9011	MSGPARSE_ERR
P9012	CTXMAP_DUP_ERR
P9013	CTXMAP_INVKEY
P9014	INVLD_TXNTYPE
P9015	INVLD_TERMINAL
P9016	DUPLICATE_TERMINAL
P9017	INVALID_PAYMENT_MODE / SECURITY_ERR
P1111	AADHAAR_TIMEOUT_ERR / DECRYPTION_FAILED
P9100	INTF_TIMEOUT_ERR
DT001	DUPLICATE_TRANSACTION
P9101	ADV_TIMEOUT_ERR
P9995	REFUND_ERR
P9996	TMPD_RESP
P9997	PROTO_ERR
P9999	INTERNAL_ERR
(none)	Internal system error — INTERNAL_ERR_DESC

Code	Description
P1001	DUP_MERREFNO
P1002	EXCEEDS_MAXVALUELIMIT
P1003	EXCEEDS_MAXVOLUMELIMIT
P1004	EXCEEDS_DAILYVALUELIMIT
P1005	MIN_TXNAMOUNT_ERR
P1006	VALIDATION_ERR
P1010	MERCHANT_EXIST
P1007	MPI_VALIDATION_ERR
P1008	INVLD_MERCHANT
P1009	ACC_NOTREGISTERED
P1011	MERCHANT_NOT_FOUND
P1012	INVALID_REQ
P1100	ORGTXN_NOTFOUND
P1101	INVLD_ORIG_STATUS
P1102	RELATED_INPROGRESS
P1103	INVLD_REM_AMOUNT
P1104	INVLD_ORIG_TXN
P1105	INVLD_AMOUNT
P1106	SAF_INPROGRESS
P1107	EXCEEDS_ACTIVITYCOUNTLIMIT
P1108	INVALID_OTP
P1109	EXCEEDS_MANDATE_ENDDATE
P1110	INVALID_VIRTUAL_ACCOUNT / INVLD_TXN
P1112	REFUND_NOT_ALLOWED
P1113	LOW_PAYABLE_BALANCE
P1000	ADV_SUCCESS — Transaction processed successfully
R1000	REQ_SUCCESS — Request processed successfully
0000	SUCCESS
ICI_57	VPA_NOT_PRESENT
A0001	ACQUIRER_INVALID_RESPONSE
A9001	ACQUIRER_GTWY_CONN_ERROR
07	INVALID_PARAMETER_VALUE
01	MISSING_PARAMETER
(none)	Internal error, contact Tech Support — DEFAULT_RESPDESC
(none)	Transaction not available in system — REQUEST_NOT_FOUND_DESC
A1011	INVALID_HASH_RESP
410	INVALID_BIN
0204	NO_RECORD
39	DEFAULT_AEPS_RESPCODE
P9500	LMS_ERROR
P9501	VOUCHER_ERROR

Code	Description
P9600	TXN_ALREADY_IN_SUCCESS_STATE
02	DEFAUL_ERROR_CODE
03	REQUEST_TYPE
S000	BBPS_REPUSH_SUCCESS
F999	BBPS_REPUSH_FAILURE
A1000	ACQUIRER_FORCE_SUCCESS
U1000	REQUEST_PENDING
E1000	EMPTY_RESPONSE
P9505	ORDER_EXPIRED
P9506	MANDATE_EXPIRED
500	ERR500
(none)	REFUND_REQ_TXN
R1001	KEEP_REFUND_TXN_REQ
U1001	REFUND_TXN_SUC_CODE_U1001
U2001	REFUND_TXN_SUC_CODE_U2001
M123	ACCEPT_COLLECT_API_CALLBACK
10008	NO_TRANSACTION_FOUND
P1115	CHARGEBACK_RAISED
P1116	REFUND_ALREADY_PROCESSED
060	TRANSACTION_TIMEDOUT
P0060	INTF_TRANSACTION_TIMEDOUT
0000	HTTP_CALL_SUCC
P8001	INVOICE_IS_NOT_FULLY_PAID_OR_NOT_CANCELLED
P8002	RISK_CHECK_FAILED_AT_DRONAPAY
S2000	SUC_DUE_TO_REJTOSUC
P1117	EMI_TSP_FAILED
P9900	NOT_SUPPORTED_PAYMENT_SUB_INSTTYPE

5. Credentials (UAT)

The following UAT credentials identify this merchant profile to the ICICI Bank Payment Gateway. Use the Key to compute the secureHash (Section 2.1) for every API call in this document.

MID	**100000000007164**
Agg ID	A100000000007164
Key	db06cca0-838b-4e01-8b20-6ac446ffb6bd
APPID	80bc18249511f868

Note: These are UAT/sandbox credentials only and must never be used against production endpoints.

6. Test Card & Test Payment Details (UAT)

Card

Use the following simulated card for card-mode payment testing on the UAT/sandbox environment.

Field	Value
Card Number	4761 3400 0000 0035
Expiry	12/26
CVV	123
OTP	123456
Name on Card	test

Net Banking

For paymentMode = NB, select the following simulated test bank on the UAT payment page.

Field	Value
Test Bank	CC Avenue Test Bank
Login / OTP	123456

UPI

For paymentMode = UPI, use the following simulated VPA to trigger a successful UAT payment.

Field	Value
Test VPA	test@ybl

Note: These are simulated UAT credentials only and must never be used in a production/live environment.

7. API URLs (UAT & Production)

Test Environment (UAT) URLs

API	UAT URL
Sale	https://pgpayuat.icicibank.com/tsp/pg/api/v2/initiateSale
Refund / Status Check	https://pgpayuat.icicibank.com/tsp/pg/api/command
Generate QR	https://pgpayuat.icicibank.com/tsp/pg/api/generateQR
Settlement Details	https://pgpayuat.icicibank.com/tsp/pg/api/settlementDetails
Settlement Summary	https://pgpayuat.icicibank.com/tsp/pg/api/command
User Cancellation API	https://pgpayuat.icicibank.com/tsp/pg/api/userCancel

Production URLs

API	Production URL
Sale	https://pgpay.icicibank.com/pg/api/v2/initiateSale
Refund / Void / Status Check	https://pgpay.icicibank.com/pg/api/command
Generate QR	https://pgpay.icicibank.com/pg/api/generateQR
Settlement Details	https://pgpay.icicibank.com/pg/api/settlementDetails
Settlement Summary	https://pgpay.icicibank.com/pg/api/command
User Cancellation API	https://pgpay.icicibank.com/pg/api/userCancel

Note: Always integrate and test fully against the UAT URLs above before switching to production hosts.

8. FAQs

1. What is the secret key for UAT?

The UAT secret key will be provided by msintegration@icici.bank.in — you need to mail this ID to request and receive the secret key for UAT testing.

2. Where do I get the secret key for production?

For production, the secret key will be provided in the Merchant Dashboard, under the Key Management section.

3. What languages and SDKs are supported?

Any language that can build a website is supported, e.g. Python, Java, PHP. SDKs are available for Flutter and Java (Kotlin), along with a WooCommerce plugin.

4. What types of integration are supported?

API integration, Seamless integration, API invoice integration, SMS invoicing, and Form Builder PG.

5. Who should we connect with for integration support?

The dedicated integration support team can be reached at msintegration@icicibank.com. For settlement and other related queries, reach out to cmssupport@icicibank.com.

6. What paymodes are available on the PG?

UPI, QR, Card, Net Banking, and Wallet. NEFT/RTGS is also supported as a paymode for net banking-based fund transfers, enabling customers to complete payments directly through their bank account via NEFT or RTGS, subject to activation for the merchant during onboarding.

7. What security certification is supported by the PG?

The PG is PCI DSS compliant. Further certification details are being confirmed. All communication between the merchant and the PG is secured using SSL encryption, and the PG enforces TLS 1.2 (or higher) for all API calls and webpage connections, ensuring data is encrypted in transit at all times.

8. Do you support server-to-server API setup, like webhooks?

Yes, webhook configuration is available in the PG. The developer needs to confirm the relevant details with us so they can be mapped to the correct MID/merchant.

9. When the transaction is in pending status, should it be marked as failed?

No — a pending transaction should not be marked as failed. Call the Status Check API to retrieve the current status, or implement the webhook URL to receive real-time updates on the transaction status. The response code for a pending transaction is P0030 (AWAITING_USER_ACTION).

10. When does the payment page expire?

The payment page does not expire on its own — it stays valid until you configure an expiry for the transaction on the backend. The payment expiry will start once you choose the payment method accordingly.